

GitHub Copilot Dev Days

From Requirements to Production
Systems: Vibe Engineering APIs
with Copilot

Hemamalini Nithyanandam
Cloud Engineering Expert – HP Inc



Agenda

A structured journey from AI fundamentals to hands-on production practices — everything you need to start building with Copilot today.

01

The Evolution of Autonomous Agents

From early IDEs to autonomous agents — understanding how we got here.

02

Agentic Workflows & Copilot Agents

Pre-agentic vs. agentic paradigms, anatomy of an AI agent, and the Coding Agent.

03

Contract-First API Development

Defining API contracts, deriving architecture, and the AI-accelerated pipeline.

04

The Full Copilot Platform

Code review, CLI, chat, SDK, multi-agent systems, and MCP.

05

Customization in Cloud Agent, Demo & Best Practices

Customizing the agent, live demo, risks, discipline, and the road ahead.

Who Am I

Professional

- Cloud Engineering Expert in HP Inc
- 13+ years in IT
- AWS Solutions Architect
- AI Solutions Engineer



<https://www.linkedin.com/in/hemamalini-nithyanandam/>

Personal

- Punnagai Foundation
- Volunteer in Gold Heart Foundation , E-Vidyaloka





IDE evolution

From Syntax Hints to Autonomous Execution

The journey has been rapid and transformative. Each generation of tooling redefined what productivity meant for engineers.

1

Early IDEs

Syntax highlighting and basic error detection — tools that helped engineers write code faster.

2

IntelliSense

Contextual suggestions, method lookups, and type inference raised the productivity ceiling significantly.

3

GitHub Copilot

AI-driven code generation from natural language — the first true leap into intent-based development.

4

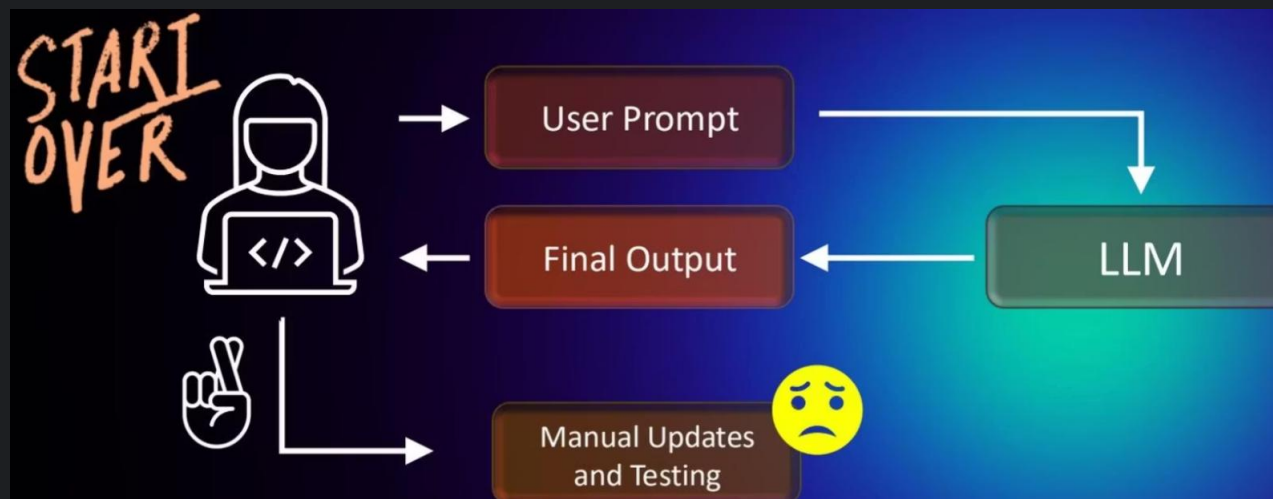
Copilot Agents

Complete task execution across files and workflows — AI that operates at the level of features, not lines.

Pre-Agentic vs. Agentic Workflows

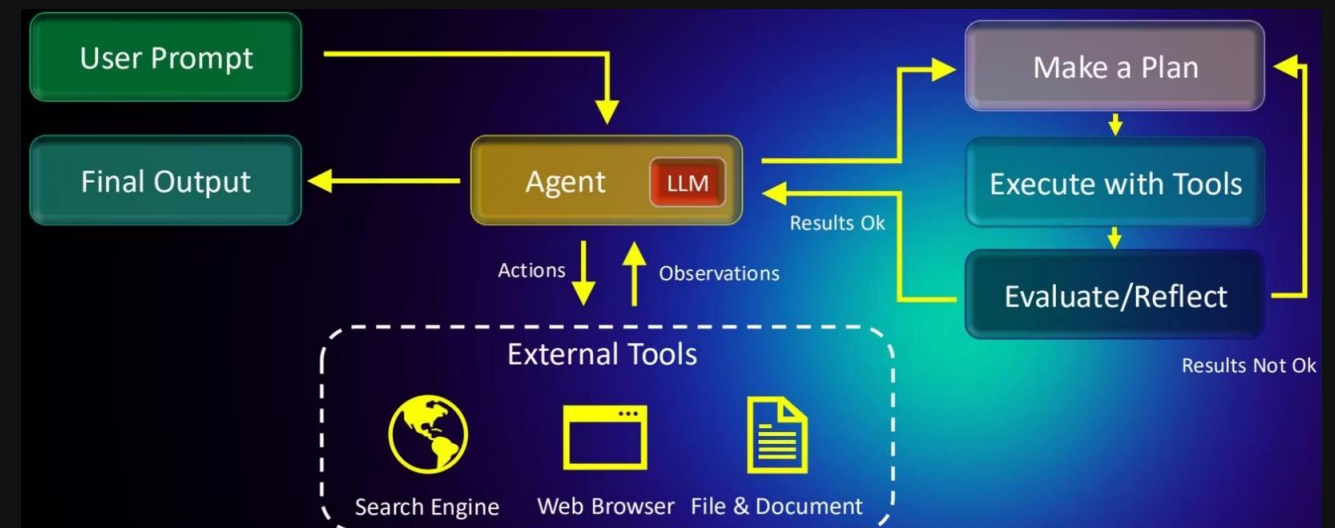
The contrast between the two paradigms is stark. In the pre-agentic world, every loop requires a human to manually re-prompt, verify, and fix. In the agentic world, the agent closes the feedback loop autonomously.

Pre-Agentic: Prompt-In, Output-Out



The developer prompts the LLM, receives output, manually tests and updates, then starts over. No feedback loop. No tool execution. High cognitive overhead.

Agentic: Plan, Execute, Evaluate



The agent receives a goal, makes a plan, executes using external tools (search, browser, files), evaluates results, and replans until done — only surfacing to the human for final review.

From Reactive to Proactive: The Agent Shift

The contrast between old and new Copilot behaviours is stark. This is not an incremental improvement — it is a categorical change in the nature of AI assistance.

Before: Copilot as Tool

- *Suggests code snippets within a single file*
- *Responds reactively to cursor position*
- *Operates at the function or block level*
- *Engineer decides every structural choice*
- *No understanding of broader repository context*

Now: Copilot as Agent

- *Executes complete workflows across multiple files*
- *Proactively plans and structures solutions*
- *Operates at the feature or task level*
- *Engineer reviews and approves, rather than authors*
- *Understands repository structure, dependencies, and conventions*



What is the GitHub Copilot Coding Agent?

*The GitHub Copilot Coding Agent is an **autonomous AI developer** that works independently in the background — completing larger, multi-step software tasks assigned through GitHub issues, chat prompts, or comments. It plans, codes, tests, and opens a PR — all without blocking your workflow.*

Coding Agent: Core Capabilities

The Coding Agent is purpose-built for end-to-end asynchronous development inside your GitHub repositories. Assign it an issue and get back a reviewed, tested pull request.

Lives In The Cloud

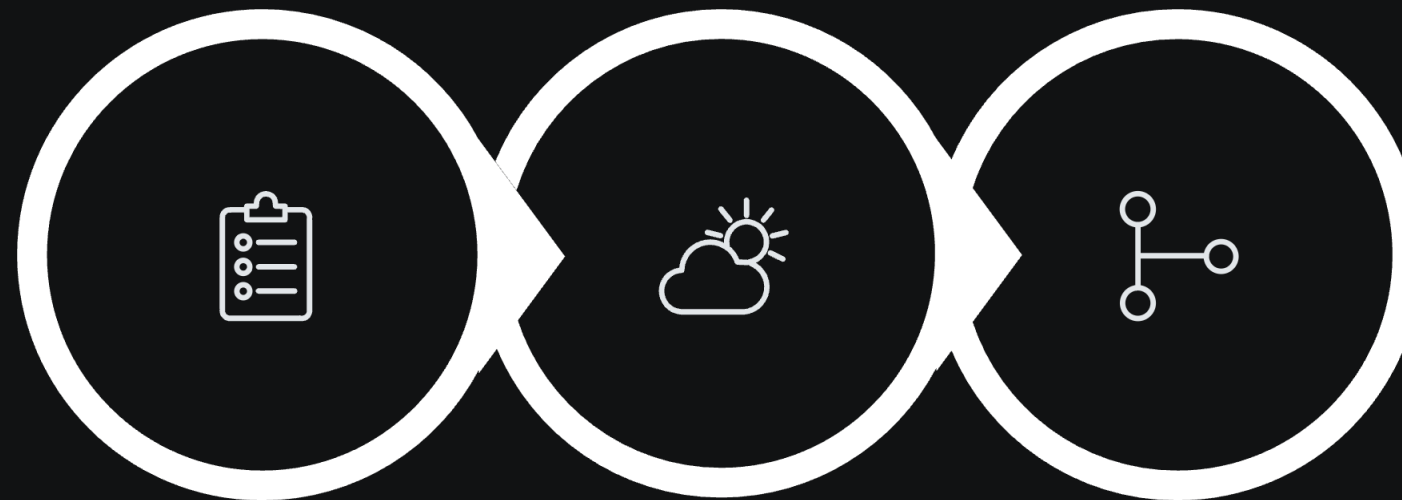
Given a GitHub issue, the agent will develop a solution, run tests and linters, and open a pull request entirely in the background — no local setup required.

Repo-Aware

Uses deep repository context to make targeted changes. Presents clean diffs, generated summaries, and well-structured PRs for fast, confident review and merge.

End-to-End Testing

Explores the codebase, makes targeted changes, and executes automated tests in an ephemeral cloud development environment — retrying on failures before surfacing results.



Define Task

Agent Runs

Review &
Ship

*The end-to-end workflow is designed to minimize interruptions
you define the goal, the agent executes autonomously, and you review the finished work before it ships.*

“ ”

“Give a dev a code completion and they'll merge once.

Teach a dev to wield an AI agent and they'll empty the backlog before the coffee cools.”

- GitHub

The Hidden Cost of Traditional Development

Before AI-native workflows, even well-run engineering teams were absorbing enormous friction. These inefficiencies were accepted as normal — but they compound across every sprint, every release, every team.

Spec-to-Code Drift

Specifications are written once and rarely revisited. By the time code ships, the implementation frequently diverges from the original design — silently and without traceability.

Code-Test Divergence

Tests are often written after the fact, if at all. They reflect what the code does rather than what it should do — inverting the entire purpose of test-driven development.

Translation Tax

Engineers spend a disproportionate amount of time translating requirements into boilerplate — work that adds little intellectual value but consumes significant capacity.

Late Validation

Review and quality gates arrive too late in the cycle. Catching architectural issues during a pull request review is expensive, disruptive, and demoralising.

Prerequisites, Access & Cost

*The Coding Agent is a **premium feature** — not included in the standard individual Copilot plan. It requires one of the following GitHub Copilot subscription tiers, and consumes premium requests and GitHub Actions minutes.*

Eligible Subscription Tiers

- Copilot Pro
- Copilot Pro+
- Copilot Business
- Copilot Enterprise

💰 Cost Considerations

*The agent uses **Copilot premium requests** and consumes **GitHub Actions minutes**, which will impact your team's monthly budget.*

- *Each plan includes a fixed number of premium requests per user per month*
- *Additional premium requests can be purchased for **\$0.04 each***
- *Set spending alerts at 75%, 90%, and 100% of your budget to avoid surprise overages*

 *Configure budget alerts in your GitHub organization settings before rolling out to your team.*

Coding Agent vs. Agent Mode

These are two distinct tools built for different moments in your workflow. Understanding when to use each — and how to use them together — is the key to maximizing developer velocity.

☁️ GitHub Copilot Coding Agent



*An **asynchronous teammate** that lives and develops entirely in the cloud. Assign it a GitHub issue and it will edit code, run tests, and open a pull request — independently, while you focus elsewhere.*

- Works from GitHub issues assigned to it
- Edits code in its own branch
- Opens a PR for your review before merging

🖥️ GitHub Copilot Agent Mode



*A **real-time collaborator** that operates directly inside VS Code. Provide a natural-language goal and it suggests code, runs tests, and answers questions — all locally, in your active session.*

- Lives inside VS Code (and other editors)
- All changes are made locally in real time
- Ideal for interactive, exploratory development

✅ *Pro Tip: You can — and should — use both agents together. Use the Coding Agent for background tasks and Agent Mode for active, in-editor sessions.*

Contract-First: A New Model

*AI-native development inverts the traditional workflow. Instead of writing code and hoping it aligns with intent, we begin with a **well-defined API contract** — and let that contract become the single source of truth for everything that follows.*

1

API Contract

Define endpoints, schemas, and validation rules with precision

2

Architecture

AI derives layered system design from the contract

3

Implementation

Code is generated consistently from the same source

4

Tests & Review

Validation artefacts are derived, not manually authored



Anatomy of a Precise API Contract

*Consider a concrete example: a **Ticket API**. Its value lies not just in defining endpoints, but in capturing enough semantic detail that an AI system can reason effectively from it.*

What the Contract Defines

- *POST /tickets — Create a new support ticket*
- *GET /tickets/{id} — Retrieve ticket by identifier*
- *Request body schemas with required fields and types*
- *Validation rules (e.g. non-empty title, valid priority enum)*
- *Error response structures with status codes*
- *Authentication and authorisation requirements*

Why Precision Matters

When a specification is vague, AI fills gaps with assumptions — and those assumptions may be wrong, insecure, or inconsistent. A contract that is precise, explicit, and machine-readable gives Copilot the grounding it needs to produce outputs that are actually useful and production-ready.



The more explicit your contract, the more reliable the AI-generated artefacts that derive from it.

From Contract to Architecture

A well-formed API specification is not just documentation — it is an architectural blueprint. Given a precise contract, Copilot can derive a coherent, layered system design with minimal manual intervention.

Layered Structure

Controllers, service classes, and repository interfaces emerge naturally from the contract's resource model and operation semantics.

Data Models

Entity definitions, DTOs, and persistence schemas are inferred from request/response bodies and validation constraints.

Validation Layer

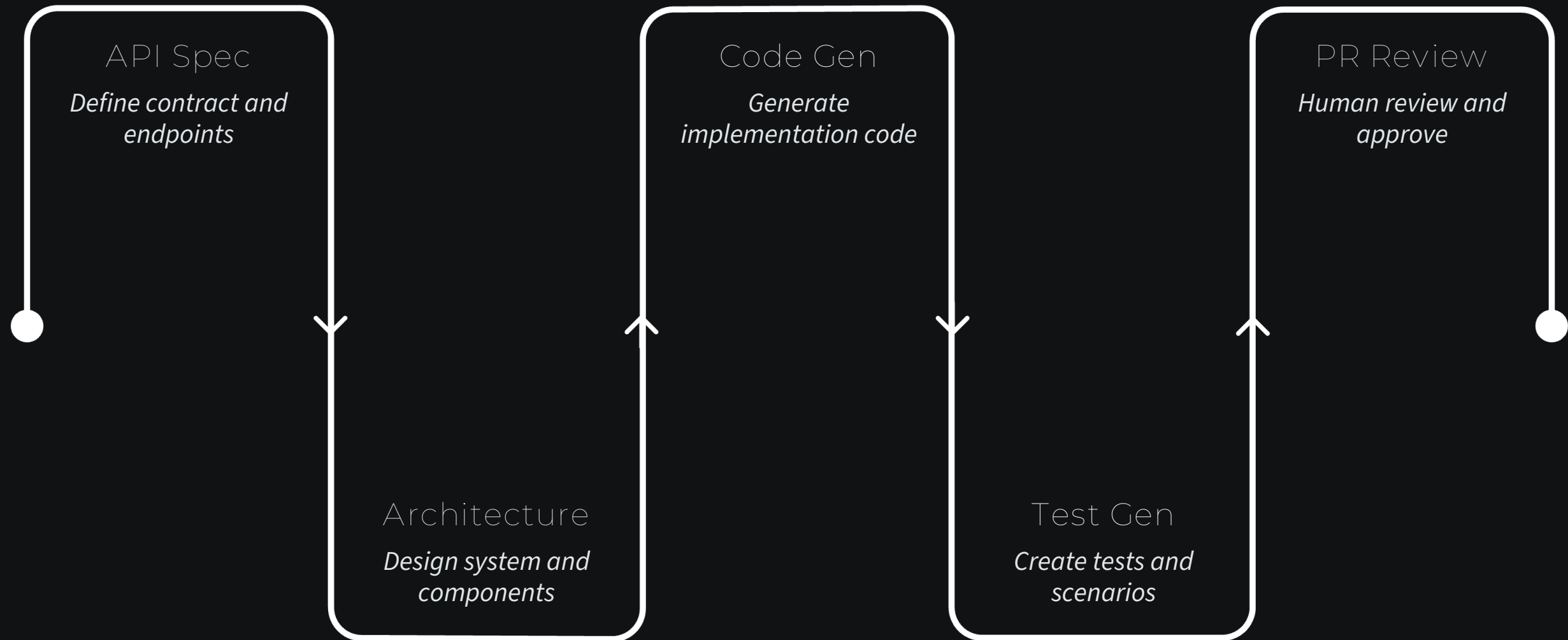
Input guards, error handlers, and constraint checks are derived directly from the rules declared in the specification.

Logging & Observability

Structured logging hooks and instrumentation points are suggested based on operation criticality and error pathways.

The AI-Accelerated Development Pipeline

The contract-first model creates a new end-to-end pipeline. Each stage flows naturally into the next, with AI accelerating the transitions while humans retain decision authority at every meaningful junction.



This pipeline does not remove engineers from the process — it removes the tedium from the process. Humans define intent at the start and validate quality at the end; AI handles the structural transformation in between.



The Copilot Agent: A New Kind of Team Member

*The most significant transformation in this evolution is the emergence of **Copilot Agents**. These are not autocomplete systems with a bigger context window. They are task-oriented collaborators that understand your repository, plan solutions, write code across multiple files, and raise pull requests — operating at the level of features, not functions.*

Copilot Agents change the unit of AI contribution from a line of code to a complete, reviewable unit of work.

A Typical AI-Native Workflow in Practice

What does a working session look like when you adopt this model? The process is more structured and deliberate — but significantly faster and more consistent than traditional approaches.

01

Author the API Specification

Write or refine an OpenAPI specification that captures endpoints, schemas, validation rules, and error contracts.

02

Derive Architecture with Copilot

Prompt Copilot to suggest a scalable architecture from the specification — layers, patterns, and module boundaries.

03

Generate Code Scaffolding

Let the agent scaffold controllers, services, repositories, and models — consistent boilerplate generated in seconds.

04

Generate Unit Tests

Prompt for test coverage across happy paths, validation failures, edge cases, and error scenarios.

05

Review, Refine, and Approve

Engineers evaluate outputs for correctness, security, and alignment with business intent before merging.

Prompting as an Engineering Discipline

*In an AI-native workflow, the quality of your output is directly proportional to the clarity of your input. **Prompting is no longer a soft skill — it is a core engineering competency**, as consequential as writing clean code or designing resilient APIs.*

Effective Prompt Principles

- **Be explicit about context** — reference the spec, framework, and target layer
- **State constraints upfront** — mention error handling, auth, and validation expectations
- **Specify output format** — define the structure you expect back
- **Iterate in layers** — start with structure, then add behaviour
- **Validate immediately** — never assume the first output is complete

Example Prompt

"Using the attached OpenAPI specification for the Ticket API, design a scalable Spring Boot backend architecture. Include a controller, service, and JPA repository layer. Apply input validation using Bean Validation and return structured error responses for 400 and 404 status codes."

What AI Generates: The Structural Layer

AI excels at producing the repetitive, pattern-driven structural components that consume disproportionate engineering time but add limited intellectual value. Freeing engineers from this work is where the productivity gains become tangible.



Controllers

Route handlers, request mapping, and HTTP response wrapping



Services

Business operation orchestration and transaction boundaries



Repositories

Data access interfaces, query methods, and persistence logic



Models

Entity definitions, DTOs, and request/response schema classes



Middleware

Authentication filters, error handlers, and logging interceptors

With this scaffolding generated automatically, engineers redirect their focus to higher-order decisions: system boundaries, resilience patterns, and domain logic that requires contextual judgment.

AI-Generated Testing: Coverage by Default

One of the most powerful applications of Copilot is generating comprehensive test suites from API specifications and implementation code. Tests are no longer an afterthought — they become a natural artefact of the contract-first workflow.

Happy Path Coverage

Valid inputs producing expected outputs — the baseline that confirms the contract is implemented correctly.

Validation Failures

Malformed requests, missing required fields, and out-of-range values — ensuring the API rejects invalid inputs gracefully.

Edge Cases

Boundary conditions, empty collections, null handling, and concurrent access scenarios that often surface late in production.

Error Scenarios

Downstream failures, timeout handling, and exception propagation — ensuring error contracts are honoured end-to-end.

The Risks You Cannot Ignore

AI-generated code is not production-ready by default. Treating Copilot as an infallible authority — rather than a capable but fallible collaborator — is one of the most dangerous patterns an engineering team can adopt.

Weak Validation Logic

Generated validators may pass on surface structure but miss business rule enforcement — e.g., accepting negative ticket priorities or empty assignee fields.

Missing Authentication

AI may scaffold endpoints without applying security annotations or middleware, silently creating unauthenticated access paths in production code.

Incorrect Assumptions

When specifications are ambiguous, the AI fills gaps with plausible-sounding but potentially incorrect logic — and does so confidently.

Prompt Injection

Maliciously crafted inputs or untrusted context injected into agent workflows can manipulate AI behaviour in unpredictable and harmful ways.

⊗ *Human oversight is not optional. Every AI-generated output must be reviewed against security, correctness, and architectural standards before merging.*

Agents Working Directly on GitHub Issues

GitHub integration transforms how features are initiated. Rather than an engineer manually translating a ticket into code, a Copilot Agent can take ownership of the entire implementation lifecycle — from issue to pull request.

The New Issue Workflow

1. *Engineer writes a GitHub issue with clear acceptance criteria*
2. *Copilot Agent reads the issue, analyses the codebase, and plans an approach*
3. *Agent generates code across relevant files, consistent with existing conventions*
4. *Agent opens a pull request explaining every change made*
5. *Engineer reviews, requests changes, or approves — focusing on quality, not authorship*

What This Changes

*The engineer's role transitions from **implementation** to **intent definition and quality gatekeeping**. Writing a great issue becomes as important as writing great code — because it is the primary input to the AI's execution.*

This shift demands clearer thinking upstream and stronger review discipline downstream.

| *"AI generates structure. Engineers own decisions."*

What AI Should — and Should Not — Own

The most effective AI-native teams operate with a clear division of responsibility. Misallocating tasks — asking AI to make architectural judgements, or asking engineers to write boilerplate — is a failure mode in both directions.

✓ AI Handles Well

- *Boilerplate code generation from contracts*
- *Data model and DTO scaffolding*
- *Unit and integration test generation*
- *Repetitive structural patterns (CRUD operations)*
- *Code formatting, documentation, and inline comments*
- *Initial pull request descriptions*

❑ Engineers Must Own

- *System architecture and module boundary decisions*
- *Security design and threat modelling*
- *Business logic and domain rule enforcement*
- *Data consistency strategies and transaction design*
- *Performance and scalability trade-offs*
- *Final review, approval, and accountability*

Where the Industry Is Heading

The trajectory is clear. The question is no longer whether AI will transform software delivery — it is how quickly teams will adapt to work alongside increasingly autonomous systems.

Autonomous Workflows

End-to-end pipelines that execute from requirement to deployment with minimal human intervention at routine stages.

Multi-Agent Systems

Specialist agents for coding, testing, security, and deployment collaborate asynchronously across the same codebase.

AI-Native Teams

Engineers who co-design with AI as a default — not as an exception — structuring their workflows around AI strengths from day one.



GitHub Copilot: Now a Full Platform

*GitHub Copilot has undergone a fundamental architectural expansion. It is no longer a single-purpose code completion feature — it is a **multi-surface, extensible AI platform** embedded across the entire software development lifecycle.*



Code Completion

Context-aware, multi-line suggestions within the editor — now significantly more accurate.



Copilot Agents

Autonomous task execution — from issue to pull request — across files, repositories, and integrated workflows.



Code Review

Automated review suggestions covering bugs, readability, and maintainability issues within pull requests.



Copilot Chat

Conversational AI with inline, repository-wide, and multi-file context awareness for design discussions and debugging.



Copilot CLI

AI-assisted command generation, script fixing, and DevOps automation directly within the terminal.

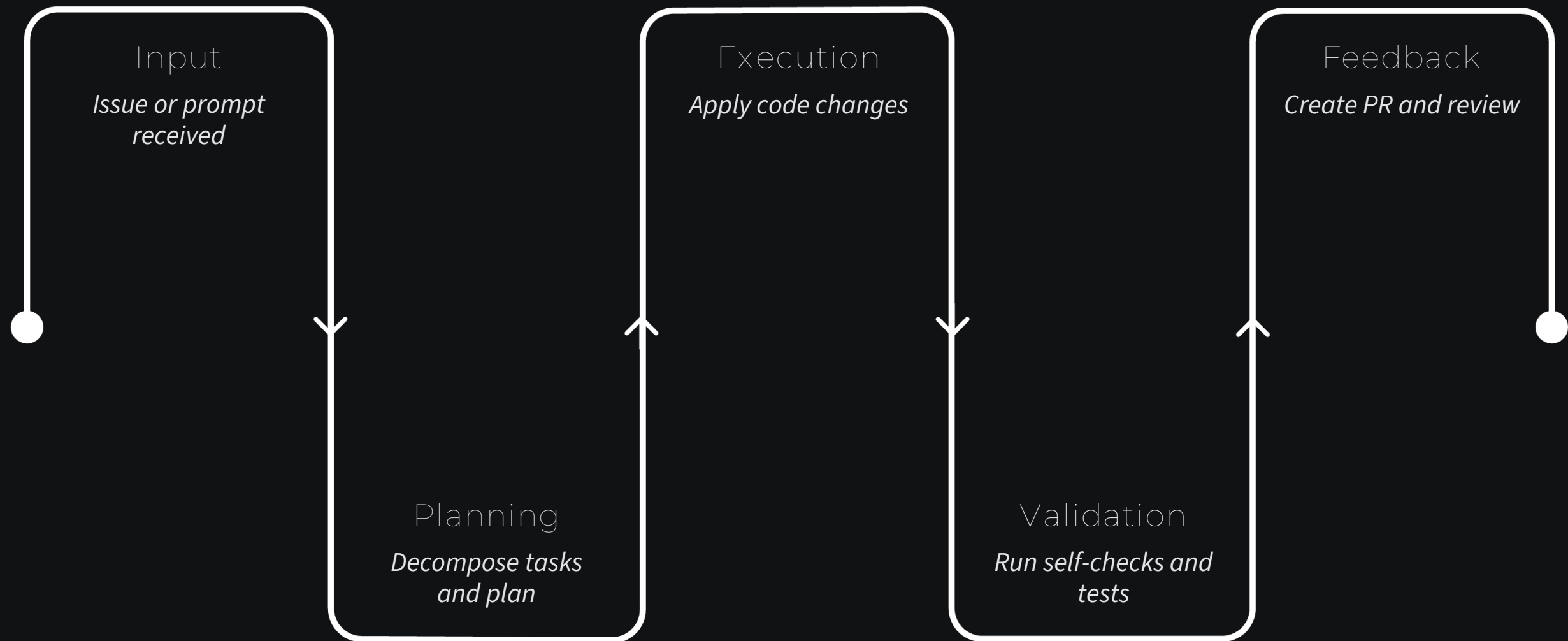


Extensible SDK

A programmable layer for building custom AI agents tailored to specific enterprise workflows and toolchains.

How Copilot Agents Execute Tasks

Copilot Agents do not just respond — they follow a structured, closed-loop execution model. Understanding this lifecycle helps engineers design better prompts, set appropriate expectations, and intervene at the right moments.



This closed-loop model means agents do not simply emit code and stop. They reason about their own outputs, check for consistency, and surface their work for human review — creating a structured handoff rather than a black-box output.

Copilot CLI: AI-Assisted DevOps in the Terminal

*For platform engineers and DevOps practitioners, **Copilot CLI** brings AI intelligence directly into the command line — the environment where infrastructure work actually happens.*



Command Generation

Describe what you need in plain language and receive a precise shell command — no more hunting through man pages for the right flags.



Script Debugging

Paste a failing Bash or Python script and get a diagnosis with a corrected version — dramatically reducing time spent on CI/CD pipeline issues.



Workflow Automation

Generate GitHub Actions YAML, Docker commands, and Kubernetes manifests from natural language descriptions of the desired behaviour.

This bridges the traditional gap between development and operations, enabling developers to contribute confidently to infrastructure and enabling ops engineers to accelerate script authoring.

Copilot Chat: Context is Everything

Copilot Chat's power scales directly with the context it can access. A shallow context produces generic responses; a deep, repository-aware context produces precise, actionable guidance.

Inline Interaction

Ask questions or request changes in context, adjacent to the specific lines of code you are working on — tight feedback loops for implementation decisions.

Multi-File Awareness

Reference multiple files in a single conversation, enabling chat to reason across service boundaries, shared utilities, and cross-cutting concerns.

Repository Understanding

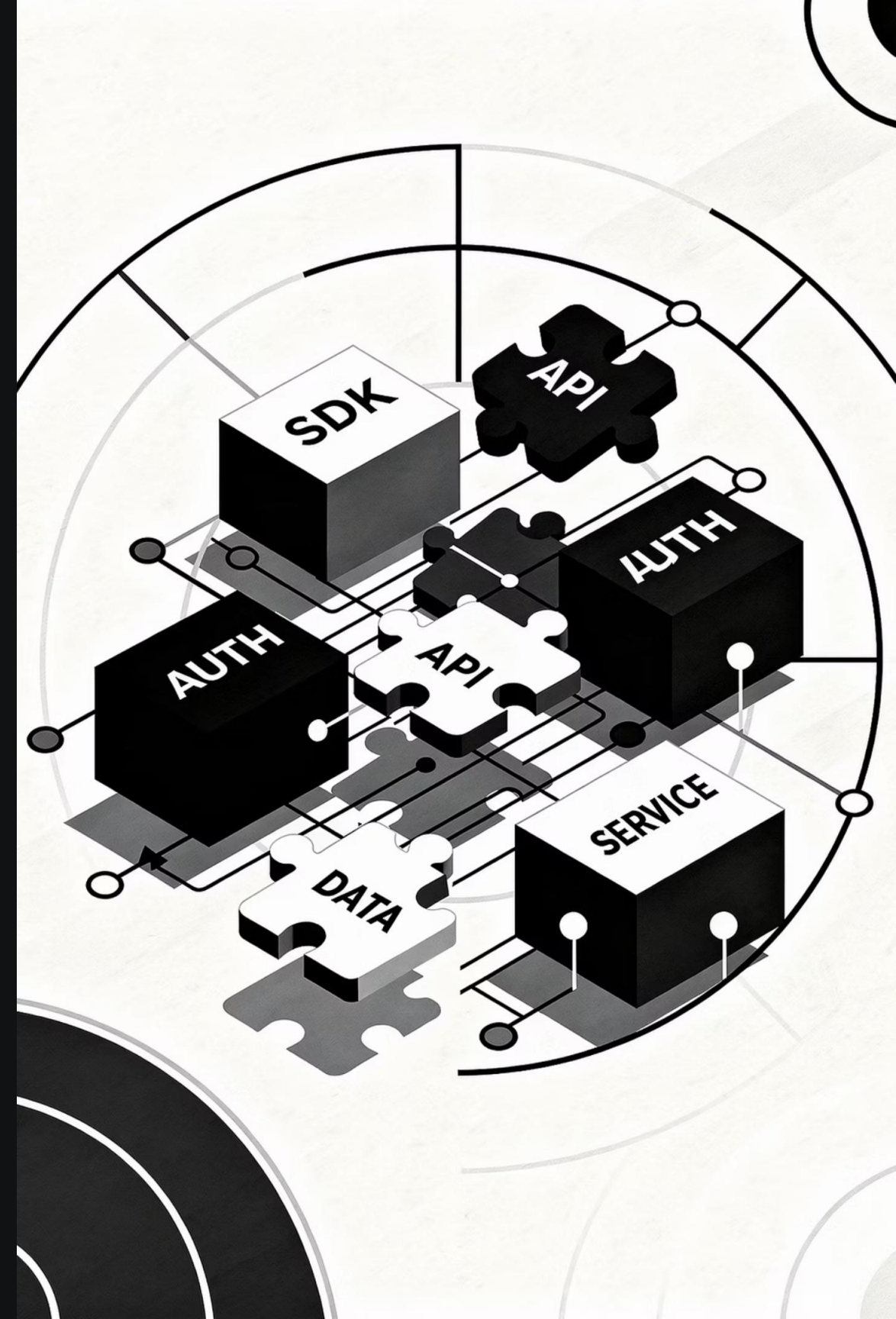
With full repository context enabled, Copilot Chat can answer questions about architecture, naming conventions, and existing patterns — acting as an always-available technical guide.

 *The investment in clear code structure, consistent naming, and good documentation pays dividends when AI is reading your repository as an input.*

The Copilot SDK: Building Custom Intelligence

*The **GitHub Copilot Extensibility SDK** is the most powerful — and most underutilised — capability in the platform. It enables engineering teams to build custom AI agents that encode domain-specific knowledge, connect to enterprise systems, and automate workflows unique to their organisation.*

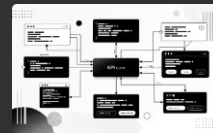
The SDK turns Copilot from a product you use into a platform you build on. This is where AI moves from productivity feature to competitive differentiator.



What You Can Build with the SDK

Custom agents built on the Copilot SDK are not generic assistants — they are purpose-built tools with domain context, system access, and workflow-specific intelligence baked in.

API Generation Agent



Takes a business requirement and produces a complete OpenAPI spec, implementation scaffolding, and validation tests — end to end.

Test Automation Agent



Analyses existing code and generates comprehensive test suites aligned to your team's testing standards and coverage targets.

DevOps Workflow Agent



Automates pipeline configuration, environment setup, and deployment scripting based on repository conventions and target infrastructure.

Security Analysis Agent



Continuously scans pull requests for OWASP patterns, dependency vulnerabilities, and policy violations before code reaches production.

Toolsets: Structuring Agent Capabilities

*A critical design pattern in the Copilot SDK is the **Toolset** — a curated grouping of capabilities that defines what an agent can access and how it may act. Toolsets prevent capability sprawl and ensure agents behave predictably within their intended scope.*

Why Toolsets Matter

- **Scope control** — agents only access the systems and operations they need
- **Predictability** — bounded capabilities mean bounded failure modes
- **Auditability** — tool invocations are logged and traceable
- **Composability** — toolsets can be combined to create multi-capability agents safely

Example: API Generator Toolset

- `read_openapi_spec` — parse and validate the input specification
- `generate_controller` — scaffold the controller layer
- `generate_tests` — produce a corresponding test suite
- `create_pull_request` — submit the artefacts for review

□ Note the absence of `delete_repository` or `modify_secrets` — toolsets enforce least-privilege by design.

Model Context Protocol: The Interoperability Layer

***Model Context Protocol (MCP)** is the emerging standard that enables agents to share context, communicate intent, and interoperate across tool boundaries. It is the infrastructure layer that makes multi-agent systems coherent rather than chaotic.*



Structured Context Sharing

Agents can pass rich, structured context — not just text strings — between themselves, preserving semantic meaning across workflow handoffs.



Tool Interoperability

MCP-compliant tools can be discovered and invoked by any compatible agent, enabling a composable ecosystem of AI capabilities.



Modular Scalability

New agents and capabilities can be added to a multi-agent system without redesigning the integration architecture — plug-and-play AI components.



Auditability

Every context exchange and tool invocation is traceable through the protocol, supporting enterprise compliance and debugging requirements.

Enterprise Adoption: Non-Negotiable Considerations

For organisations operating at scale — particularly in regulated industries — AI adoption is not simply a technical decision. It requires deliberate governance frameworks that address data, access, compliance, and accountability.

Data Privacy

Understand what code and context is transmitted to AI models. Evaluate data residency, retention policies, and whether proprietary algorithms are used in training.

Access Control

Apply least-privilege principles to agent toolsets. Agents should not have write access to production systems, secrets, or infrastructure without explicit, audited authorisation.

Compliance

Map AI-assisted development practices to relevant frameworks — SOC 2, ISO 27001, GDPR. Ensure AI-generated code is covered by your existing review and approval gates.

Auditability

Log every agent invocation, tool call, and generated artefact. Maintain a clear chain of custody from AI output to production deployment for accountability and incident response.

  first commit

amerejess commented just now

@copilot can you continue where I left off? Just need to
and do the busywork



Copilot commented just now

I've opened a new pull request, [#3624](#), to work on this. When
ready, I'll request review from you.



 **Copilot** linked an issue that may be closed by this PR:
[WIP]: Add test coverage and update glowing

COPILOT CODING AGENT

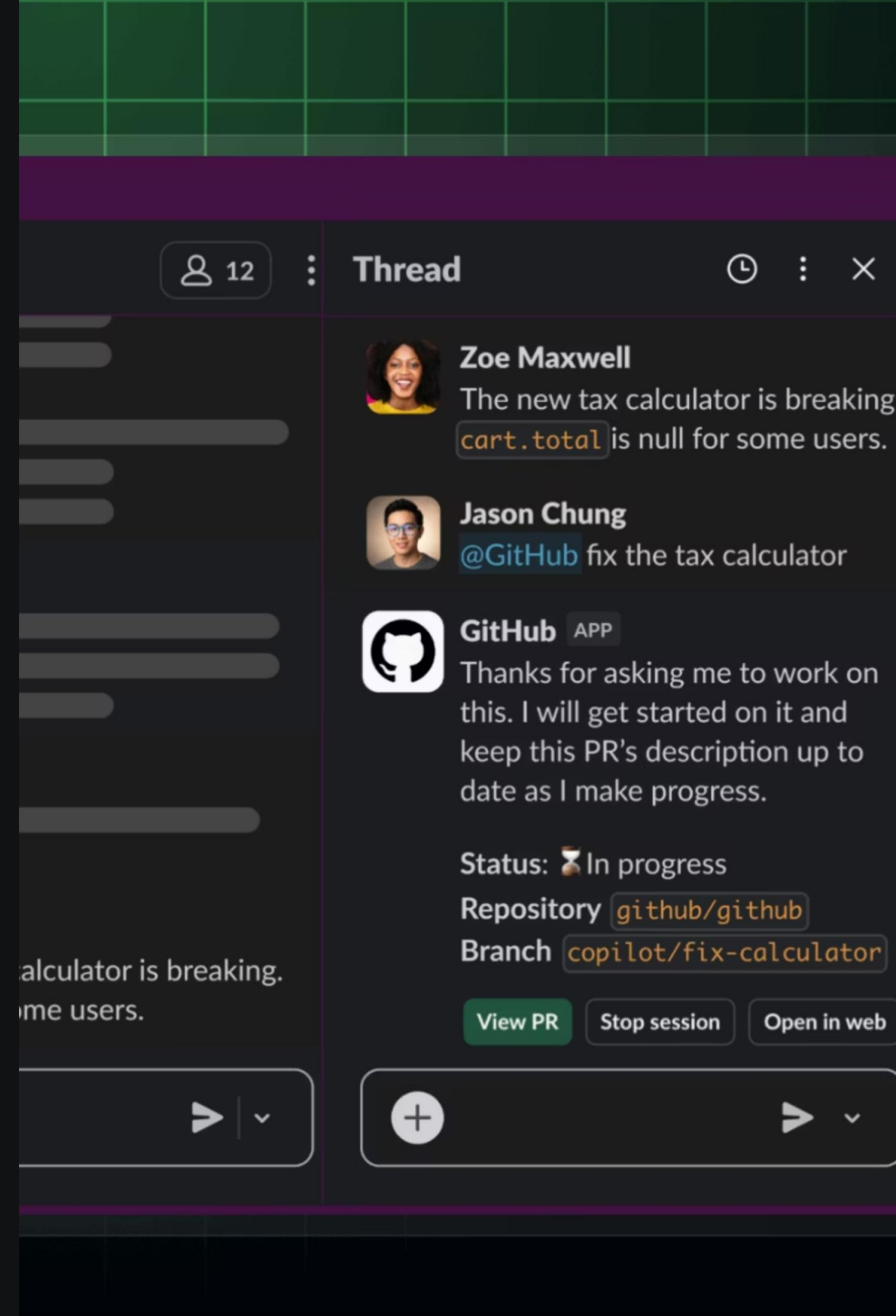
Ask Copilot to make changes to any PR

Copilot is no longer limited to opening new PRs. Mention @copilot in an existing PR to have Copilot make changes.

SLACK INTEGRATION

Start Copilot coding agent tasks from Teams or Slack

Go from collaboration to code without skipping a beat. Just mention @GitHub in any conversation.



Assign Linear issues to Copilot

Hand Linear issues to Copilot in one click, with



Install GitHub Copilot in Linear

⚠️ **Admin access required:** Only Linear workspace admins can install this integration.

What this integration does:

This integration enables your team to assign Linear issues to GitHub Copilot, an AI coding agent that can work on development tasks automatically.

- ✓ Adds a "GitHub Copilot" agent to your Linear workspace
- ✓ Allows team members to assign issues to the agent
- ✓ Enables GitHub account linking for individual developers
- ✓ Processes development tasks using GitHub Copilot

Installation steps:

1. Click the install button below to authorize Linear access
2. Grant the necessary permissions in Linear
3. The agent will appear in your workspace as "GitHub Copilot"
4. Team members can now assign issues to the agent

Cancel

Install in Linear

Installing this integration will redirect to:
linear.app/xvz

Owned & operated
by GitHub

Created
1 month ago

Used by
more than 1k users



GitHub Copilot installed successfully!

✓ **Great news!** GitHub Copilot has been successfully installed in your linear workspace.

The integration is now active for your organization:

ID: 2a930117-6a1e-4a7b-a59e-cf42f314d5a3

What's next?

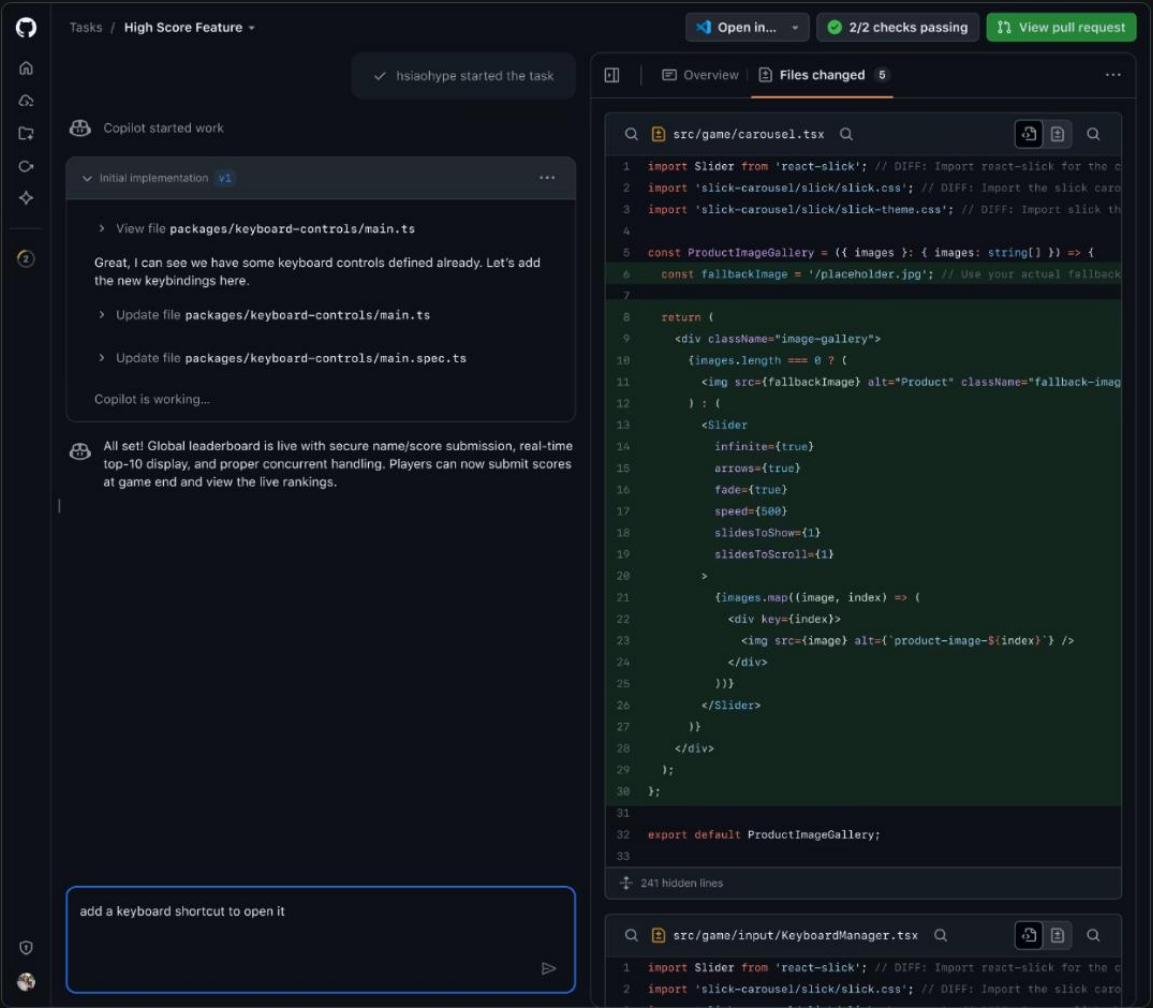
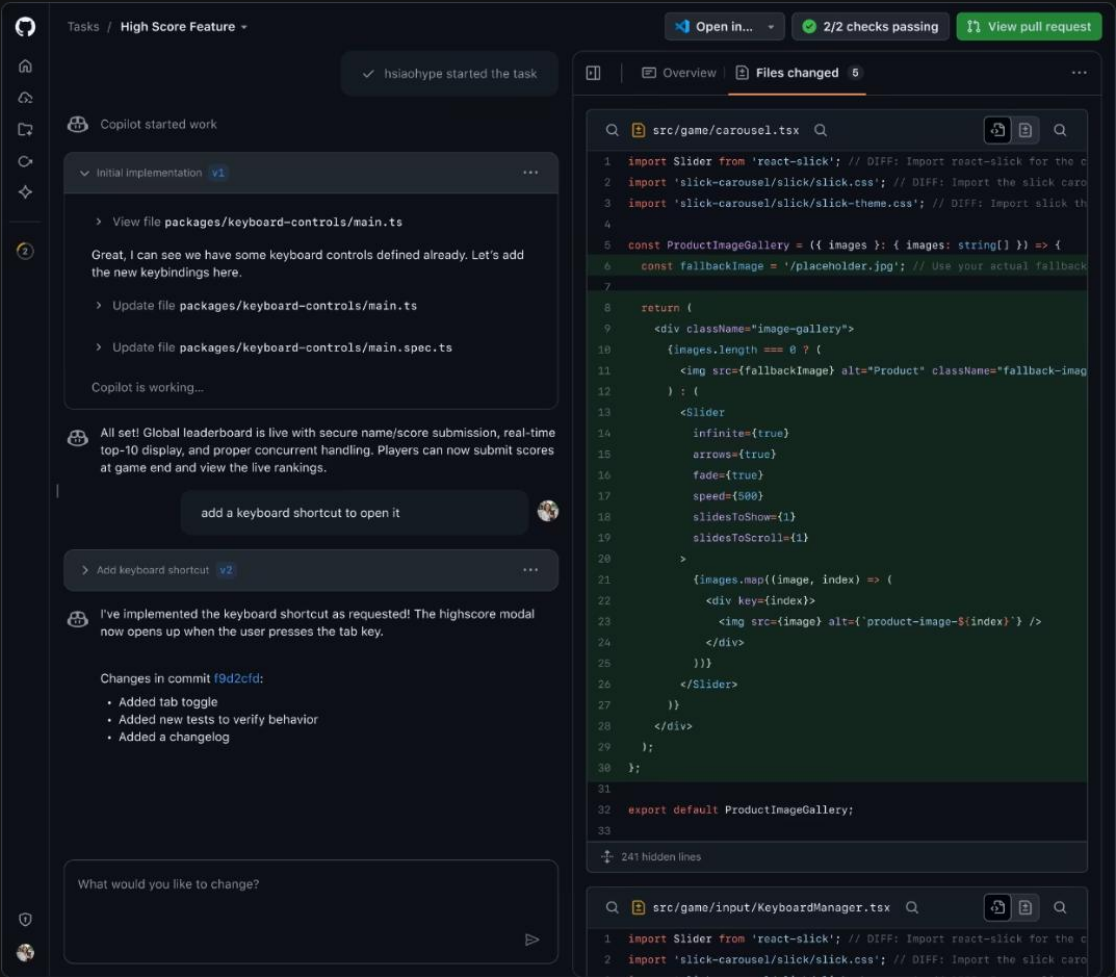
To start using GitHub Copilot on your issues:

1. Go to any issue in your Linear workspace
2. Assign the issue to **GitHub Copilot**
3. The agent will guide you through linking your GitHub account (first-time only)
4. Then GitHub Copilot will automatically start working on your issue!
 - The agent processes issues and creates pull requests automatically
 - Progress updates are posted as comments on the Linear issue

Close window and return to Linear [↗](#)

Manage, monitor, steer and review agent sessions, all in one place

Jump between sessions, see what's changed, track Copilot's progress, and step in with course-corrections as needed.



Validation tools

Stay secure while you move faster

Copilot checks its work automatically before asking for review, using the power of CodeQL, Secret Scanning, the GitHub Advisory Database and Copilot code review

Perfect! The tests found no issues. Now let me run the security checks.

› Check new dependencies against the GitHub Advisory Database

Excellent! No vulnerabilities in the new dependencies. Let me now run the CodeQL security scan.

✓ Run CodeQL security scan on changed files

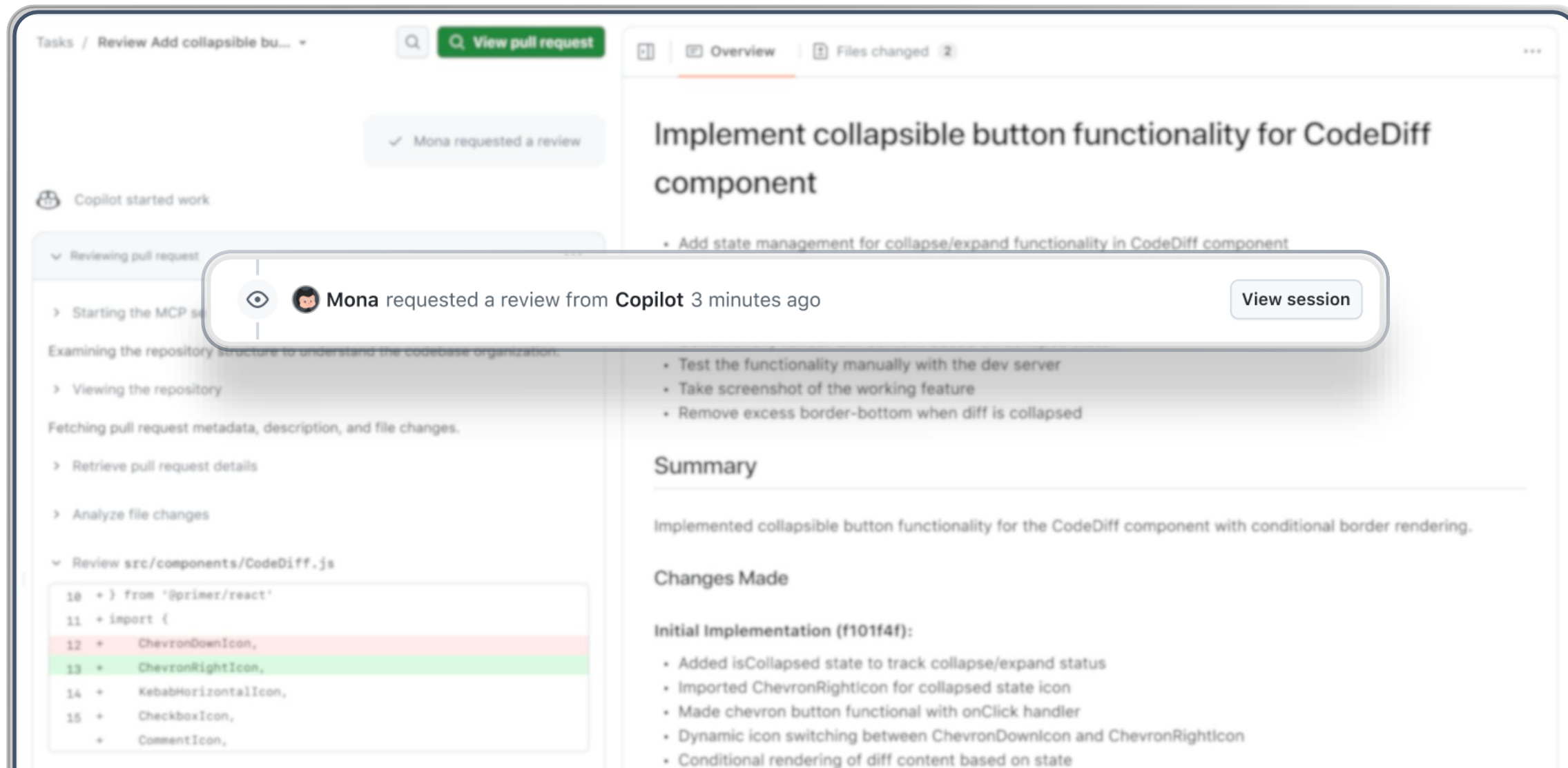
Analysis Result for 'actions, javascript'. Found 0 alert(s):

- actions: No alerts found.
- javascript: No alerts found.

Perfect! No security vulnerabilities found. Let me create a final summary.

Copilot code review

Have Copilot review code, then hand updates to coding agent for implementation



CODING AGENT

Customizing Copilot Coding Agent

`copilot-instructions.md`

Repository and organization level instructions that coding agent always considers — shape its behaviour to match your team's standards and conventions.

`copilot-setup-steps.yaml`

Customize the environment that coding agent runs in — install dependencies, configure access, and prepare the workspace before work begins.

MCP tools

Configure MCP servers that Copilot can use to bring in outside context — databases, APIs, and enterprise tools become first-class inputs.

Custom Agents

Define a custom agent that is an expert in a specific area — reducing repetitive prompting and improving consistency across recurring tasks.

copilot-instructions.md

Create repository custom instructions files that give Copilot additional context.

Repository custom instructions let you provide Copilot with repository-specific guidance and preferences.

Project structure

- server/ : Flask backend code
 - models/ : SQLAlchemy ORM models
 - routes/ : API endpoints organized by resource
 - tests/ : Unit tests for the API
 - utils/ : Utility functions and helpers, including database calls
- client/ : Astro/Svelte frontend code
 - src/components/ : Reusable Svelte components
 - src/layouts/ : Astro layout templates
 - src/pages/ : Astro pages and routes
 - src/styles/ : CSS stylesheets
- scripts/ : Development, deployment and testing scripts
- docs/ : Project documentation to be kept in sync at all times


```

name: "Copilot Setup Steps"

# Automatically run the setup steps when they are changed
# Allows for streamlined validation,
# and allow manual testing through the repository's "Actions" tab

on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml
  pull_request:
    paths:
      - .github/workflows/copilot-setup-steps.yml

jobs:
  # The job MUST be called `copilot-setup-steps`
  # otherwise it will not be picked up by Copilot.
  copilot-setup-steps:
    runs-on: ubuntu-latest

    # Permissions set just for the setup steps
    # Copilot has permissions to its branch

    permissions:
      # To allow us to clone the repo for setup
      contents: read

    # The setup steps – install Python and our dependencies
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python

```

copilot-setup-steps.yml

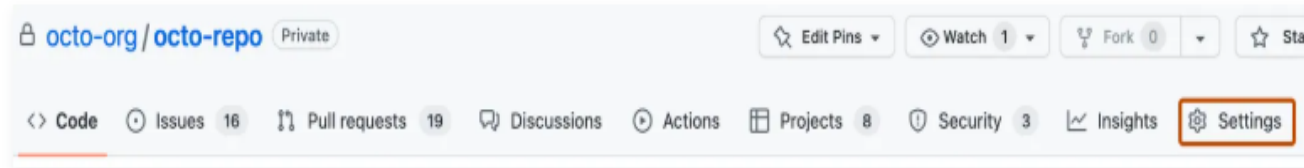
Configure Copilot's environment with GitHub Actions

You can preconfigure Copilot's environment before the agent starts by creating a special GitHub Actions workflow file, located at `.github/workflows/copilot-setup-steps.yml` within your repository.

Adding an MCP configuration to your repository [↗](#)

Repository administrators can configure MCP servers by following these steps:

- 1 On GitHub, navigate to the main page of the repository.
- 2 Under your repository name, click  **Settings**. If you cannot see the "Settings" tab, select the ... dropdown menu, then click **Settings**.



- 3 In the "Code & automation" section of the sidebar, click **Copilot** then **Cloud agent**.
- 4 Add your configuration in the **MCP configuration** section.

The following sections in this article explain how to write the JSON configuration that you need to enter here.

- 5 Click **Save**.

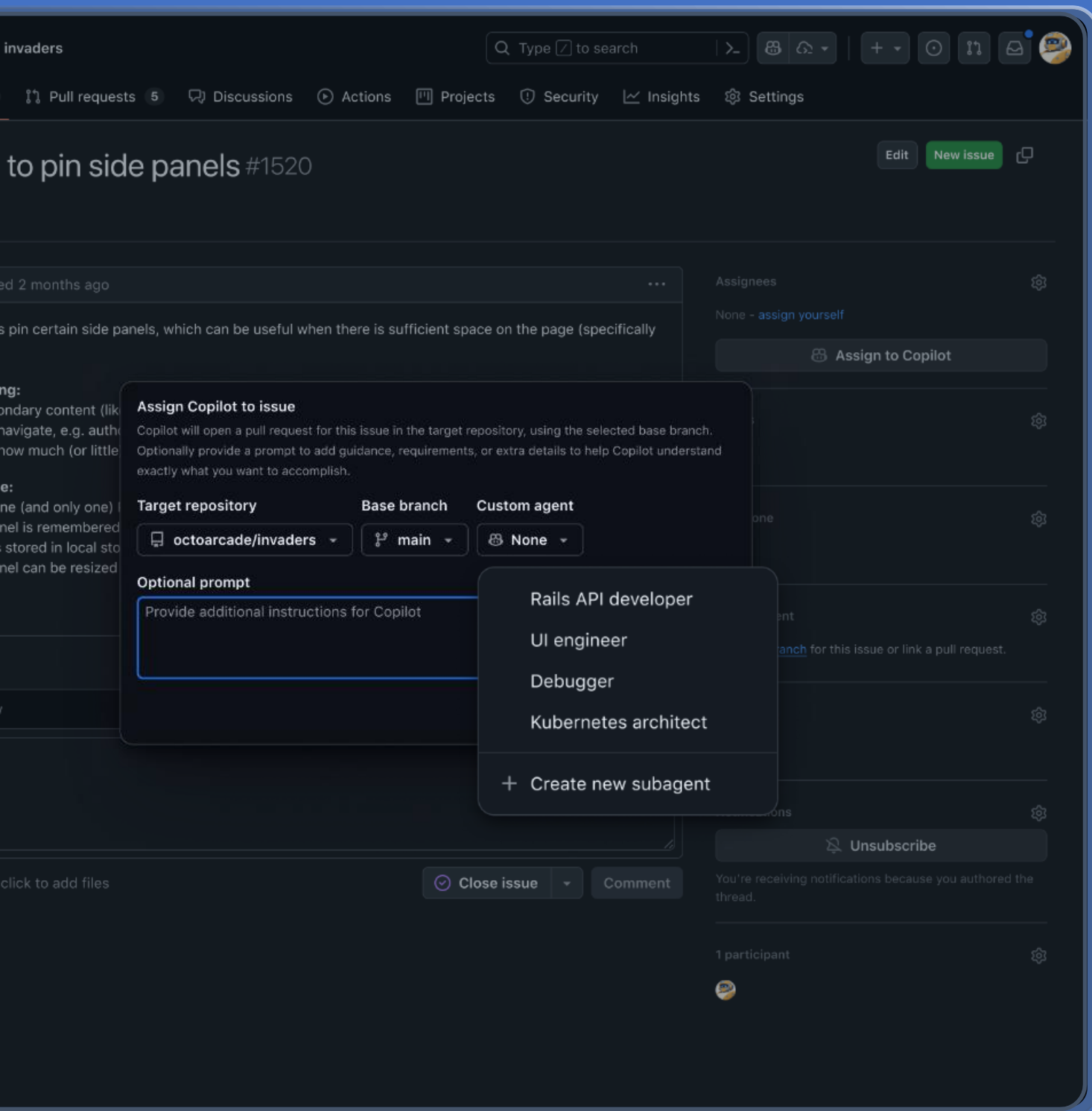
Your configuration will be validated to ensure proper syntax.

- 6 If your MCP server requires a variable, key, or secret, add a variable or secret to your Copilot environment. Only variables and secrets with names prefixed with `COPILOT_MCP_` will be available to your MCP configuration. See [Setting up a Copilot environment for Copilot cloud agent](#).

MCP Servers

Configure MCP servers to access additional context

Provide Copilot access to more tools, store tokens using environment secrets.



Custom agents

Reduce repetitive prompting by creating specialized agents to target recurring tasks

Equipped with tailored prompts, tool sets, and MCP server connections, you can implement features or fix bugs without needing to configure every time

github-arcade-s-hope

Private

generated from [github/spark-template](#)

Watch 0

Fork 0

Star 0

Your main branch isn't protected

Protect this branch from force pushing or deletion, or require status checks before merging. [View documentation.](#)

[Dismiss](#)[Protect this branch](#)

main

8 Branches

0 Tags

Go to file

t

+

<> Code

timrogers	Delete package-lock.json	848c87c · 2 days ago	🕒 5 Commits
📁 .github	Initial commit	2 days ago	
📁 src	Generated by Spark: Add more dynamic a...	2 days ago	
📄 .gitignore	Initial commit	2 days ago	
📄 .spark-initial-sha	Generated by Spark: Arcade-styled app f...	2 days ago	
📄 LICENSE	Initial commit	2 days ago	
📄 PRD.md	Generated by Spark: Arcade-styled app f...	2 days ago	
📄 README.md	Initial commit	2 days ago	
📄 SECURITY.md	Initial commit	2 days ago	
📄 components.json	Initial commit	2 days ago	
📄 index.html	Generated by Spark: Arcade-styled app f...	2 days ago	

About

No description, website, or topics provided.

- Readme
- MIT license
- Security policy
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

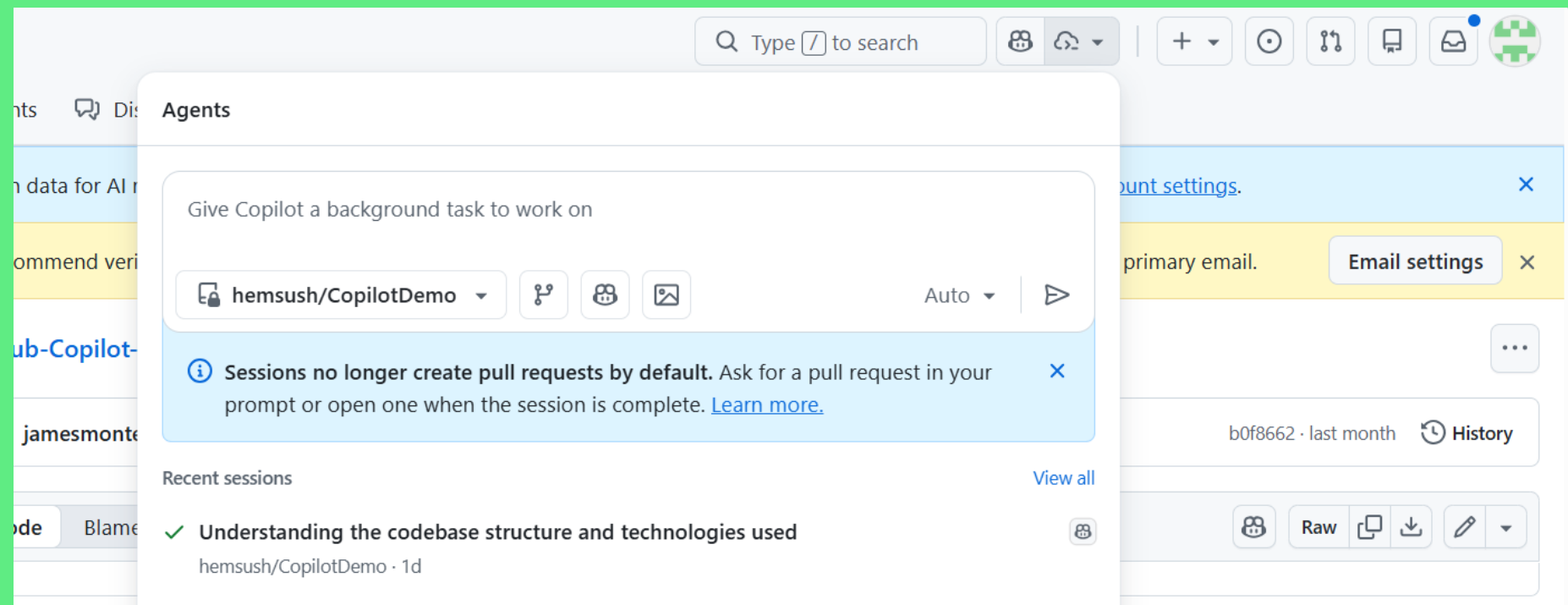
Packages

No packages published
[Publish your first package](#)

Languages



Demo



Honest Assessment: Copilot's Limitations

Sustainable adoption of AI in engineering requires intellectual honesty about what these systems cannot reliably do. Over-trusting AI outputs is a failure mode that manifests slowly — and expensively.



Hallucination

Copilot can generate plausible-looking code that references non-existent methods, incorrect APIs, or outdated library versions — confidently and without warning.



Incomplete Logic

Generated implementations often handle the happy path well but miss edge cases, concurrency scenarios, or business rule exceptions that require domain knowledge.



Security Blind Spots

Without explicit security prompting, AI-generated code routinely omits authentication checks, input sanitisation, and secure defaults.

The Right Mental Model

*Treat Copilot as a **highly productive junior engineer**: fast, knowledgeable about patterns, but requiring supervision for anything touching security, architecture, or business-critical logic.*

*The goal is **accelerated delivery with maintained quality standards** — not unreviewed automation.*

Engineering Discipline: Best Practices for AI-Native Teams

The teams extracting the most value from Copilot are not those who use it most aggressively — they are those who use it most deliberately.

Discipline remains the differentiator between teams that accelerate and teams that accumulate AI-generated technical debt.

01

Write Strong API Specifications

The contract is the foundation. Invest time in precision — explicit validation rules, clear error contracts, and versioning strategy. A weak spec produces weak outputs at every subsequent stage.

03

Validate Every Output

Review AI-generated code against your security checklist, architecture guidelines, and business rules before merging. Never assume correctness.

02

Give Clear, Contextual Prompts

Reference the specification, the framework, the target layer, and the constraints. Treat prompt-writing as code review — it deserves the same rigour.

04

Iterate Incrementally

Generate in layers: structure first, then behaviour, then edge cases. Smaller, focused iterations are easier to validate and correct than large, monolithic outputs.

The Future: Autonomous Software Delivery

*The trajectory of AI-native development points toward a future where **continuous, autonomous software delivery** becomes the operating model — not the exception. Engineers who develop fluency with AI collaboration now will be the architects of that future.*

1

Near Term

AI agents handle routine implementation, freeing senior engineers for architecture and security. Review velocity increases as AI pre-screens PRs.

2

Medium Term

Multi-agent pipelines autonomously manage feature delivery for well-scoped work. Humans set standards, approve releases, and handle exceptions.

3

Long Term

AI-first teams where engineers operate as system designers, intent definers, and quality governors — collaborating with specialised AI agents permanently.

THE FUTURE OF ENGINEERING

We Are Not Replacing Developers. We Are Upgrading How Development Happens.

Copilot started as a pair programmer.

It suggested the next line. It completed the obvious pattern. It was a faster autocomplete — useful, but bounded.

Now it is becoming a team member.

It plans tasks. It reads issues. It writes across files. It opens pull requests. It reviews code. It executes workflows.

*The engineers who thrive in this era are those who **design intent clearly, validate outputs rigorously, and direct AI with the same precision they once applied to writing code.** The craft of software engineering is not diminished by AI. It is elevated — to a higher level of abstraction, and a higher standard of outcomes.*

Q&A



<https://github.com/hemsush/GitHubCopilotDevDay2026>

Further Links to Explore

Continue your GitHub Copilot learning journey with these curated resources — from official documentation to hands-on courses and video walkthroughs.

Official Documentation

GitHub Copilot Agents documentation — how-tos, API references, and configuration guides for the full coding agent platform.

docs.github.com — Use Copilot Agents

Udemy Course

A comprehensive hands-on course covering the GitHub Copilot Coding Agent from setup to advanced workflows.

udemy.com — GitHub Copilot Coding Agent

YouTube Walkthrough

Video walkthrough demonstrating the end-to-end contract-first development pipeline with Copilot in action.

youtube.com — Copilot Demo Walkthrough

Session Repository

All code, specs, and agent configurations from this GitHub Copilot Dev Days session.

github.com/github/GitHub-Copilot-Dev-Days